

Trabajo de asd

María Torres Chacón
Pablo Gamero García

L4 Grupo 2



Índice

Descripción del problema

Análisis de código

Grafo de dependencias

OpenMP

MPI

OpenMP + MPI

Conclusiones

Descripción de la plataforma :

Cluster de la
asignatura.

- procesador:** 32 x Inter(R) Xeon(R) CPU E5-2660 @ 2.20GHz (2 Sockets)
- memoria RAM:** 188.83GiB
- SO:** Linux 6.8 12-20-pve (2026-03-13T08:15Z)
- compilador:** gcc 13.3.0
- ensamblador:** ghidra 12.0.4

Explicación del problema

FUNCIÓN

Algoritmo de fuerza bruta para descifrar contraseñas.

NUESTRA CONTRASEÑA EJEMPLO:

“asd123”

-6 caracteres => tiempo de ejecución máximo y factible.

CARACTERÍSTICAS

Descifra contraseñas con:

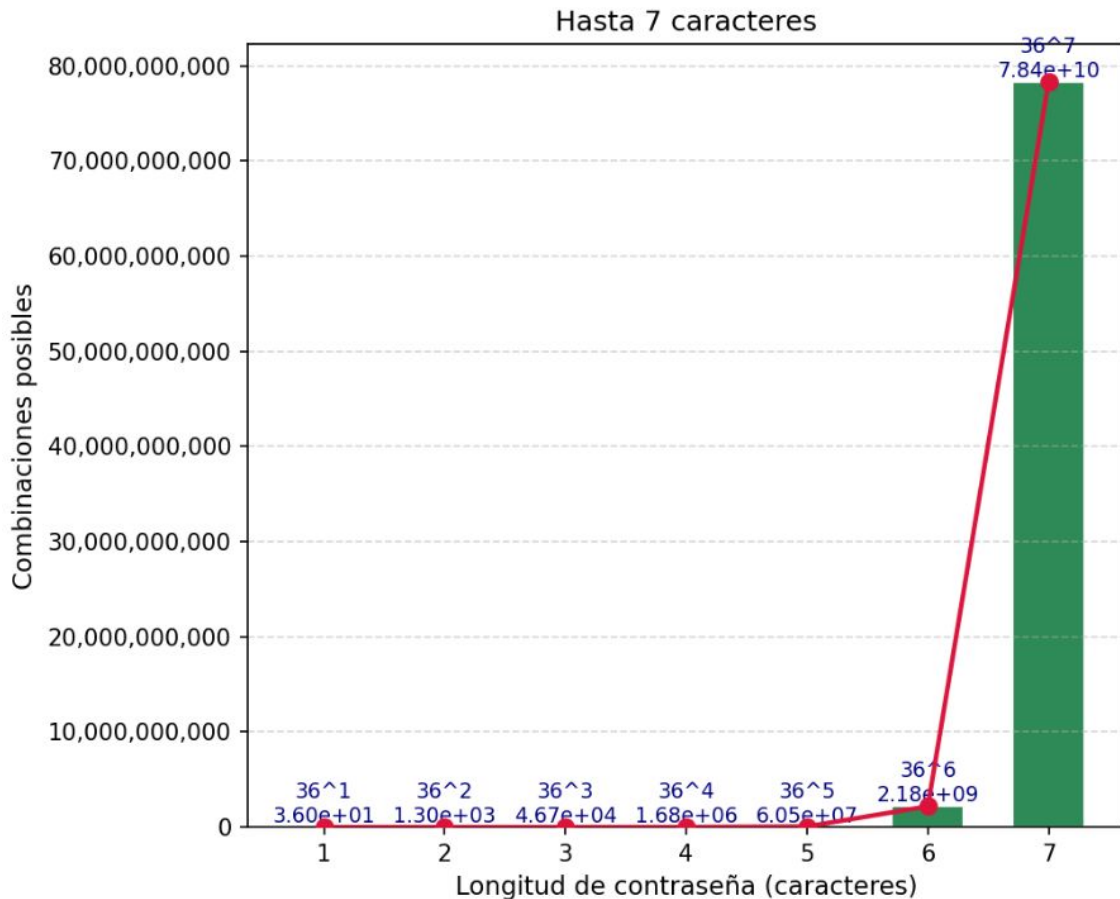
- 36 caracteres alfanuméricos posibles:

-26 MINÚSCULAS

-10 NÚMEROS

Convierte a base 36 para iterar combinaciones.

Combinaciones posibles con 36 caracteres posibles según longitud



-Longitud 1 = 36

-Longitud 2 = 1.296

-Longitud 3 = 46.656

-Longitud 4 = 1.679.616

-Longitud 5 = 60.466.176

-Longitud 6 = 2.176.782.336

-Longitud 7 = 78.364.164.096

-Longitud 8 = 82.821.109.907.456

COMPLEJIDAD Y ASPECTOS A MEJORAR

Espacio de búsqueda crece forma exponencial en función de **n** (longitud de la contraseña).

- **COMPLEJIDAD: $O(36^n)$** , cada carácter extra multiplica el trabajo por 36.

En nuestro ejemplo “**asd123**”: **2176 Millones de combinaciones.**

- **OBJETIVO: Reducir tiempo de ejecución.**

Mucha capacidad de mejora dividiendo el espacio de búsqueda entre hilos y procesos, además de usando opciones de compilación.

CÓDIGO: importaciones, constantes y medición de tiempo

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>
#include <time.h>

#define CONTRASENA "asd123"
#define LONGITUD_CONTRASENA 6
#define CARACTERES "abcdefghijklmnopqrstuvwxyz0123456789"
#define NUM_CAR 36

static double tiempo_monotonico_segundos(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec + ((double)ts.tv_nsec * 1e-9);
}
```

CÓDIGO: funciones auxiliares

```
//función que sirve para convertir un indice numerico a una cadena,  
// para poder probar todas las posibles combinaciones  
void indice_a_cadena(long long indice, int longitud, char *resultado){  
    for (int i = longitud - 1; i >= 0; i--) {  
        resultado[i] = CARACTERES[indice % NUM_CAR];  
        indice = indice / NUM_CAR;  
    } resultado[longitud] = '\0';  
}
```

```
//por que pow es para double  
static long long potencia_entera(int base, int exponente) {  
    long long resultado = 1;  
    for (int i = 0; i < exponente; i++) {  
        resultado *= base;  
    }  
    return resultado;  
}
```

Explicación índice_a_cadena

Convierte un número (índice) a base 36.

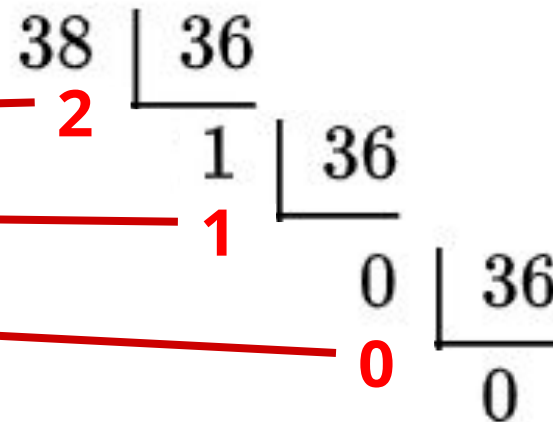
EJEMPLO: **CARACTERES** = "abcdefghijklmnopqrstuvwxyz0123456789"

Longitud de contraseña = 3

CARACTERES[2] = "c"

CARACTERES[1] = "b"

CARACTERES[0] = "a"



38 = "abc"

CÓDIGO: main I - declaraciones

```
int main(void) {  
    printf("BUSCANDO CONTRASENA: '%s' CON LONGITUD: %d, CARACTERES  
POSIBLES: %d)\n", CONTRASENA, LONGITUD_CONTRASENA, NUM_CAR);  
    printf("MEDICION CON CLOCK_MONOTONIC (segundos)\n\n");  
  
    // NUM_CAR elevado a LONGITUD_CONTRASENA  
    long long total_combinaciones = potencia_entera(NUM_CAR,  
LONGITUD_CONTRASENA);  
  
    printf("POSIBLES COMBINACIONES: %lld\n", total_combinaciones);  
  
    int encontrado = 0;  
    char resultado[LONGITUD_CONTRASENA + 1];  
    encontrado = 0;  
    double start = tiempo_monotonico_segundos();
```

CÓDIGO: main II - probar combinaciones (código a optimizar)

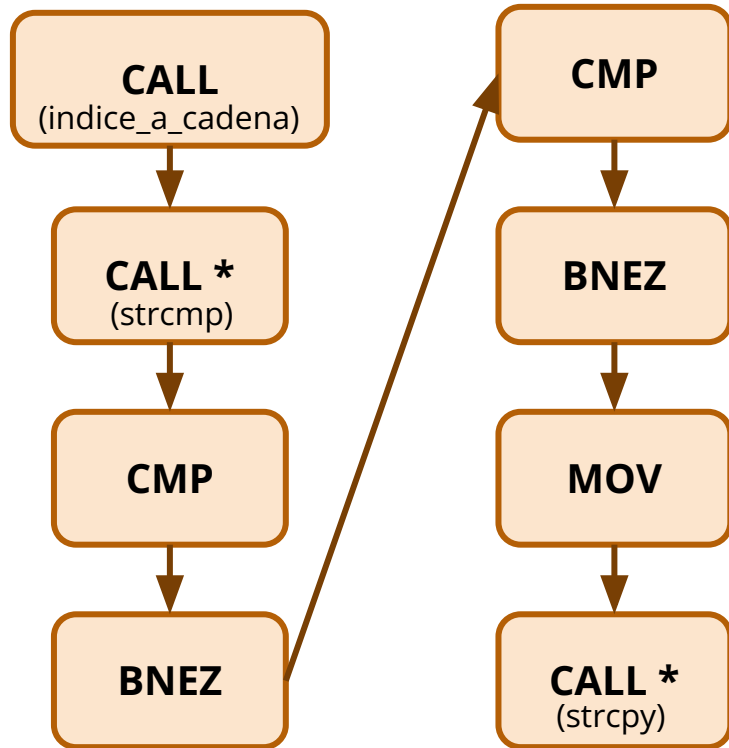
```
for (long long i = 0; i < total_combinaciones; i++) {  
    char cadena[LONGITUD_CONTRASENA + 1];  
    indice_a_cadena(i, LONGITUD_CONTRASENA, cadena);  
  
    if (strcmp(cadena, CONTRASENA) == 0) {  
        if (encontrado == 0) {  
            encontrado = 1;  
            strcpy(resultado, cadena);  
        }  
    }  
}  
  
double end = tiempo_monotonico_segundos();  
double segundos = end - start;
```

CÓDIGO: main II - comprobación de si se encontró la contraseña

```
if (encontrado) {  
    printf("CONTRASENA ENCONTRADA: '%s'\n", resultado);  
} else {  
    printf("NO SE ENCONTRO\n");  
}  
  
printf("TIEMPO (s): %.6f\n", segundos);  
  
return 0;  
}
```

GRAFO DE DEPENDENCIAS - main

Iteración 1

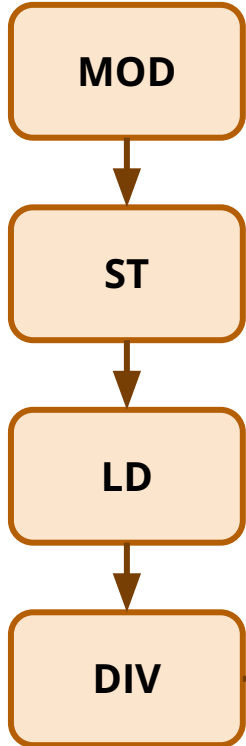


***strcmp:** Lee byte a byte cada elemento de ambos arrays y los compara (hasta encontrar el `"/0"`).

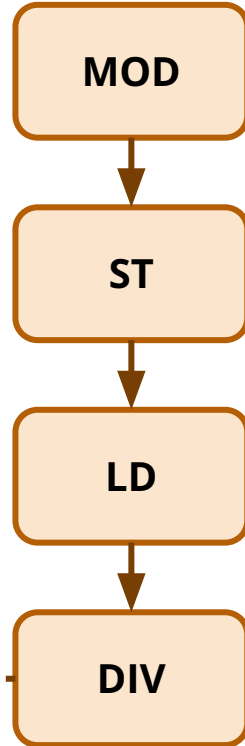
***strcpy:** Copia cada byte de un array a otro hasta array (hasta llegar a `\0`).

GRAFO DE DEPENDENCIAS - Indice_a_cadena

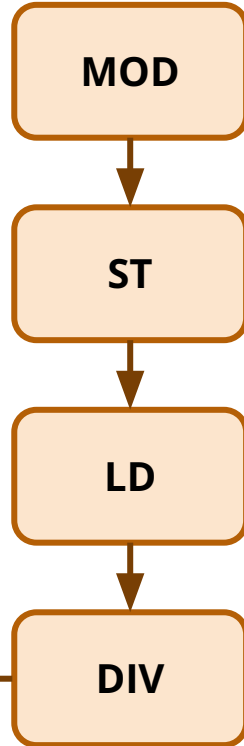
Iteración 1



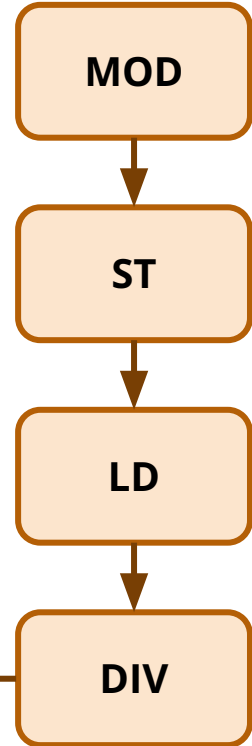
Iteración 2



Iteración 3



Iteración 4



LEY DE AMDAHL

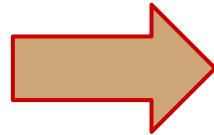
$$A = \frac{T}{(1 - \sigma)T + \frac{\sigma}{p}T} = \frac{1}{(1 - \sigma) + \sigma/p}$$

$$\sigma = 0.99$$

-El bucle (mayor tiempo de ejecución) no tiene dependencia loop-carried

Porcentaje de código paralelizable: 99%

$$A(p) = \frac{1}{(1 - 0.99) + \frac{0.99}{p}}$$



p	Aceleración
2	1.98
4	3.8835
8	7.4738
10	9.1743

Intensidad aritmética

```
void indice_a_cadena(long long indice, int longitud, char *resultado){  
    for (int i = longitud - 1; i >= 0; i--) {  
        resultado[i] = CARACTERES[indice % NUM_CAR];  
        indice = indice / NUM_CAR;  
    } resultado[longitud] = '\0';  
}
```

IOPS = 2 x 6 iter.

ACCESOS = 2 x 6 iter.

***Con
operaciones
enteras**

BYTES x ACCESOS = 1 (char)

Intensidad aritmética

```
for (long long i = 0; i < total_combinaciones; i++) {  
    char cadena[LONGITUD_CONTRASENA + 1];  
    indice_a_cadena(i, LONGITUD_CONTRASENA, cadena);  
  
    if (strcmp(cadena, CONTRASENA) == 0) {  
        if (encontrado == 0) {  
            encontrado = 1;  
            strcpy(resultado, cadena);  
        }  
    }  
}
```

Al ser constante no es necesario accederlo en memoria cada vez

IOPS = 0

ACCESOS = 12 + 1

***Con
operaciones
enteras**

BYTES x ACCESOS = 6 (6 char)

IA/BW

En nuestro código:

$$IA = \text{iOPs} / \text{BYTES LEÍDOS}$$

-SOLO OPERACIONES ENTERAS

Adaptando la IA para operaciones enteras (IOPS):

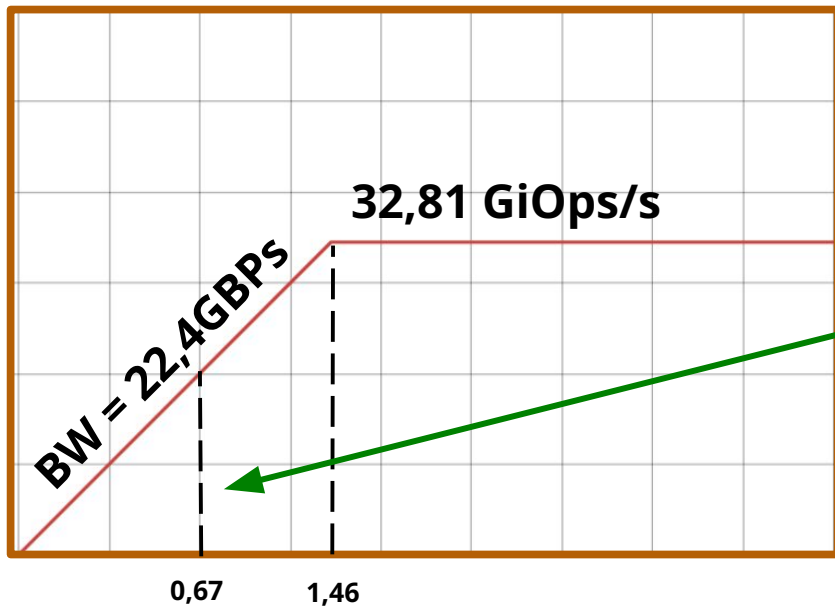
$$IA = \frac{12 \text{ IOPS}}{(12 \text{ Accesos} * 1 \text{ Byte}) + (1 \text{ Acceso} * 6 \text{ Bytes})} = 0.67$$

$$IA/BW = \frac{0.67}{22.4} = 0.03$$

MODELO DEL TEJADO

*cpu del cluster: Integer Math, 32.810 MOps/Sec.

$$IA = \text{GFLOPs} / \text{BYTES LEÍDOS}$$



LIMITACIÓN: ANCHO DE BANDA.

Cuello de botella: memory-bound

METODOLOGÍA

```
// compile_all_optimizations.sh
```

```
for opt in "${OPT_LEVELS[@]}"; do
```

```
    flag="-${opt}"
```

```
    echo "Compilando con $flag..."
```

```
    gcc "$flag" -march=x86-64-v2 "$SEQ_SRC" -o "${SEQ_DIR}/cracker_${opt}"
```

```
    mpicc "$flag" -march=x86-64-v2 "$MPI_SRC" -o "${MPI_DIR}/cracker_MPI_${opt}"
```

```
    gcc "$flag" -march=x86-64-v2 -fopenmp "$OMP_SRC" -o "${OMP_DIR}/cracker_OMP_${opt}"
```

```
    mpicc "$flag" -march=x86-64-v2 -fopenmp "$OMP_MPI_SRC" -o
```

```
    "${OMP_MPI_DIR}/cracker_OMP_MPI_${opt}"
```

```
done
```

Comandos y scripts utilizados para compilar

Técnicas de paralelización usadas

1. O3
2. OpenMP
3. MPI
4. OpenMP + MPI
5. OpenMP + O3
6. MPI + O3
7. OpenMP + MPI + O3

***O3 -> DPL + ILP**

Opción -O3

Algunas opciones de compilación de O3.

ILP

Loop unrolling

Reorganización de código

Predicción de saltos

DLP

Auto-vectorización SIMD

Opción instrucciones SIMD:
-march=x86-64-v2.

incluye juegos de instrucciones:

**SSE3, SSSE3, SSE4.1, SSE4.2,
POPCNT**

Juegos de instrucciones SIMD

- Varios juegos. El compilador usa el que sea conveniente.

Registros de 128 bits.

nuestras cadenas => 6 bytes (48 bits).

Juegos con más bits **no aportan mejora.**

Lo más beneficioso:

- Operaciones vectoriales.
- Optimización de comparación de cadenas de caracteres (strcmp)



DLP

Auto-vectorización SIMD

Opción instrucciones SIMD:
-march=x86-64-v2.

incluye juegos de instrucciones:

**SSE3, SSSE3, SSE4.1, SSE4.2,
POPCNT**

Código generado (00) - indice_a_cadena

00101207 48 8b 4d e8 **MOV** **RCX,qword ptr [RBP + local_20]**

0010120b 48 ba 39 **MOV** **RDX,0xe38e38e38e38e39**

00101215 48 89 c8 **MOV** **RAX,RCX**

00101218 48 f7 ea **IMUL** **RDX**

0010121b 48 d1 fa **SAR** **RDX,0x1**

0010121e 48 89 c8 **MOV** **RAX,RCX**

00101221 48 c1 f8 3f **SAR** **RAX,0x3f**

00101225 48 29 c2 **SUB** **RDX,RAX**

00101228 48 89 d0 **MOV** **RAX,RDX**

0010122b 48 c1 e0 03 **SHL** **RAX,0x3**

0010122f 48 01 d0 **ADD** **RAX,RDX**

00101232 48 c1 e0 02 **SHL** **RAX,0x2**

00101236 48 29 c1 **SUB** **RCX,RAX**

00101239 48 89 ca **MOV** **RDX,RCX**

➔ **INDICE%36**

Código generado (00) - indice_a_cadena

```
0010123c 8b 45 fc    MOV     EAX,dword ptr [RBP + local_c]
0010123f 48 63 c8    MOVSXD  RCX,EAX
00101242 48 8b 45 d8    MOV     RAX,qword ptr [RBP + local_30]
00101246 48 01 c1    ADD     RCX,RAX  posición de memoria de resultado[i]

00101249 48 8d 05    LEA     RAX,[s_abcdefghijklmnopqrstuvwxyz012345_001020 =
"abcdefghijklmnopqrstuvwxyz012  puntero base CARACTERES[]

00101250 0f b6 04 02  MOVZX   EAX=>s_abcdefghijklmnopqrstuvwxyz012345_001020 =
"abcdefghijklmnopqrstuvwxyz012  CARACTERES[indice % 36]

00101254 88 01      MOV     byte ptr [RCX],AL
mete CARACTERES[indice % 36] en resultado[i]
```

Código generado (00) - indice_a_cadena

00101256 48 8b 4d e8 MOV RCX,qword ptr [RBP + local_20]

0010125a 48 ba 39 MOV RDX,0xe38e38e38e38e39

00101264 48 89 c8 MOV RAX,RCX

00101267 48 f7 ea IMUL RDX

0010126a 48 89 d0 MOV RAX,RDX

0010126d 48 d1 f8 SAR RAX,0x1

00101270 48 c1 f9 3f SAR RCX,0x3f

00101274 48 89 ca MOV RDX,RCX

00101277 48 29 d0 SUB RAX,RDX

0010127a 48 89 45 e8 MOV qword ptr [RBP + local_20],RAX

0010127e 83 6d fc 01 SUB dword ptr [RBP + local_c],0x1

INDICE/36

Código generado (00) - for del main

```
001013f7 48 8d 55 f1    LEA    RDX=>local_17,[RBP + -0xf]  Dirección memoria de cadena
001013fb 48 8b 45 c0    MOV    RAX,qword ptr [RBP + local_48]
001013ff be 06 00    MOV    ESI,0x6    Longitud contraseña
00101404 48 89 c7    MOV    RDI,RAX
00101407 e8 dd fd    CALL   indice_a_cadena      undefined indice_a_cadena()
```

```
0010140c 48 8d 45 f1    LEA    RAX=>local_17,[RBP + -0xf]  Dirección memoria de cadena
00101410 48 8d 15    LEA    RDX,[s_asd123_0010202d] = "asd123"  Dirección memoria de
contraseña
00101417 48 89 d6    MOV    RSI=>s_asd123_0010202d,RDX  = "asd123"      contraseña
0010141a 48 89 c7    MOV    RDI,RAX      cadena
0010141d e8 ce fc    CALL   <EXTERNAL>::strcmp  int strcmp(char * __s1, char * _)
```

Código generado (00) - for del main

```
00101422 85 c0    TEST    EAX,EAX
00101424 75 20    JNZ     LAB_00101446
```

if (strcmp == 0)

```
00101426 83 7d bc 00    CMP     dword ptr [RBP + local_4c],0x0
0010142a 75 1a    JNZ     LAB_00101446
```

if (encontrado == 0)

```
0010142c c7 45 bc    MOV     dword ptr [RBP + local_4c],0x1
```

Salta el bloque según la condición

```
00101433 48 8d 55 f1    LEA     RDX=>local_17,[RBP + -0xf]
00101437 48 8d 45 ea    LEA     RAX=>local_1e,[RBP + -0x16]
0010143b 48 89 d6    MOV     RSI,RDX
0010143e 48 89 c7    MOV     RDI,RAX
00101441 e8 5a fc    CALL    <EXTERNAL>::strcpy      char * strcpy(char * __dest, cha
```

busca posiciones de memoria de cadena
y resultado para usar el strcpy

Código generado (03) - indice_a_cadena

001014ac 49 b8 39	MOV	R8,0xe38e38e38e38e39	Una vez al inicio del bucle
001014b6 4a 8d 74	LEA	param_2,[param_3 + R10*0x1 + -0x2]	
001014bb 48 01 d7	ADD	param_1,param_3	
001014be 4c 8d 0d	LEA	R9,[s_abcdefghijklmnopqrstuvwxyz012345_0010200 = "abcdefghijklmnopqrstuvwxyz012	

No carga constantemente los mismos datos de memoria, guarda una vez en registros y los reutiliza

Código generado (03) - indice_a_cadena

LAB_001014d0

XREF[1]: 00101500(j)

001014d0 48 89 c8	MOV	RAX,RCX
001014d3 48 83 ef 01	SUB	param 1,0x1

001014d7 49 f7 e8	IMUL	R8
001014da 48 89 c8	MOV	RAX,RCX
001014dd 48 c1 f8 3f	SAR	RAX,0x3f
001014e1 48 d1 fa	SAR	param_3,0x1
001014e4 48 29 c2	SUB	param_3,RAX

reutiliza para
división y
módulo

001014e7 48 8d 04 d2	LEA	RAX,[RDX + RDX*0x8]
001014eb 48 c1 e0 02	SHL	RAX,0x2
001014ef 48 29 c1	SUB	RCX,RAX

módulo

Código generado (03) - indice_a_cadena

001014f2 41 0f b6 MOVZX EAX,byte ptr [R9 + RCX*0x1]=>s_abcdefghijklmno =
"abcdefghijklmnopqrstuvwxyz012

CARACTERES[indice % 36]

001014f7 48 89 d1 MOV RCX,param_3

resultado[i] = CARACTERES[indice % 36]

001014fa 88 47 01 MOV byte ptr [param_1 + 0x1],AL ç

Actualiza siguiente indice (indice/36)

Código generado (03) - for del main

001011f3 4c 89 f8 MOV RAX,R15

→ 001011f6 49 f7 e4 **MUL R12**

1 001011f9 48 89 d6 MOV RSI,RDX

001011fc 4c 89 fa MOV RDX,R15

001011ff 48 c1 ee 05 SHR RSI,0x5

00101203 48 8d 04 f6 LEA RAX,[RSI + RSI*0x8]

00101207 48 c1 e0 02 SHL RAX,0x2

0010120b 48 29 c2 SUB RDX,RAX

0010120e 48 89 f0 MOV RAX,RSI

00101211 45 0f b6 MOVZX R9D,byte ptr [R14 + RDX*0x1]=>DAT_200000000010

→ 00101216 49 f7 e4 **MUL R12**

00101219 48 89 d1 MOV RCX,RDX

2 0010121c 48 c1 e9 05 SHR RCX,0x5

00101220 48 8d 04 c9 LEA RAX,[RCX + RCX*0x8]

00101224 48 c1 e0 02 SHL RAX,0x2

..... **(HASTA 6 VECES)**

El compilador omitió la llamada a Indice_a_cadena

Desenrolló 6 veces (una por carácter)

Código generado (03) - for del main

```
00101288 c1 e0 08    SHL     EAX,0x8
0010128b 09 d0          OR      EAX,EDX
0010128d 41 0f b6    MOVZX   EDX,byte ptr [R14 + RDI*0x1]=>DAT_7fe2ee564f1d
00101292 c1 e0 08    SHL     EAX,0x8
00101295 09 d0          OR      EAX,EDX
00101297 41 0f b6    MOVZX   EDX,byte ptr [R14 + RSI*0x1]=>DAT_fc6c201f0136
0010129c c1 e0 08    SHL     EAX,0x8
0010129f 09 d0          OR      EAX,EDX
001012ab 44 88 4c    MOV     byte ptr [RSP + local_4b],R9B
001012b0 44 88 54    MOV     byte ptr [RSP + local_4c],R10B
001012b5 89 44 24 28  MOV     dword ptr [RSP + local_50],EAX
001012b9 c6 44 24    MOV     byte ptr [RSP + local_4a],0x0
```

**mete 4 caracteres en un solo registro =>
reduce número de accesos a memoria**

Cálculo de parámetros (Secuencial)

Secuencial + O3 vs. Secuencial + O0

Aceleración obtenida

$$\frac{\textit{Tiempo secuencial}}{\textit{Tiempo Optimizado}}$$

$$104,5159355 / 44,9594371 = \mathbf{2,324671627}$$

Eficiencia

$$\frac{\textit{Tiempo secuencial}}{\textit{cores}}$$

$$2,324671627 / 1 = \mathbf{2,324671627} \text{ (o } 232,467162\%)$$

OpenMP (Código)

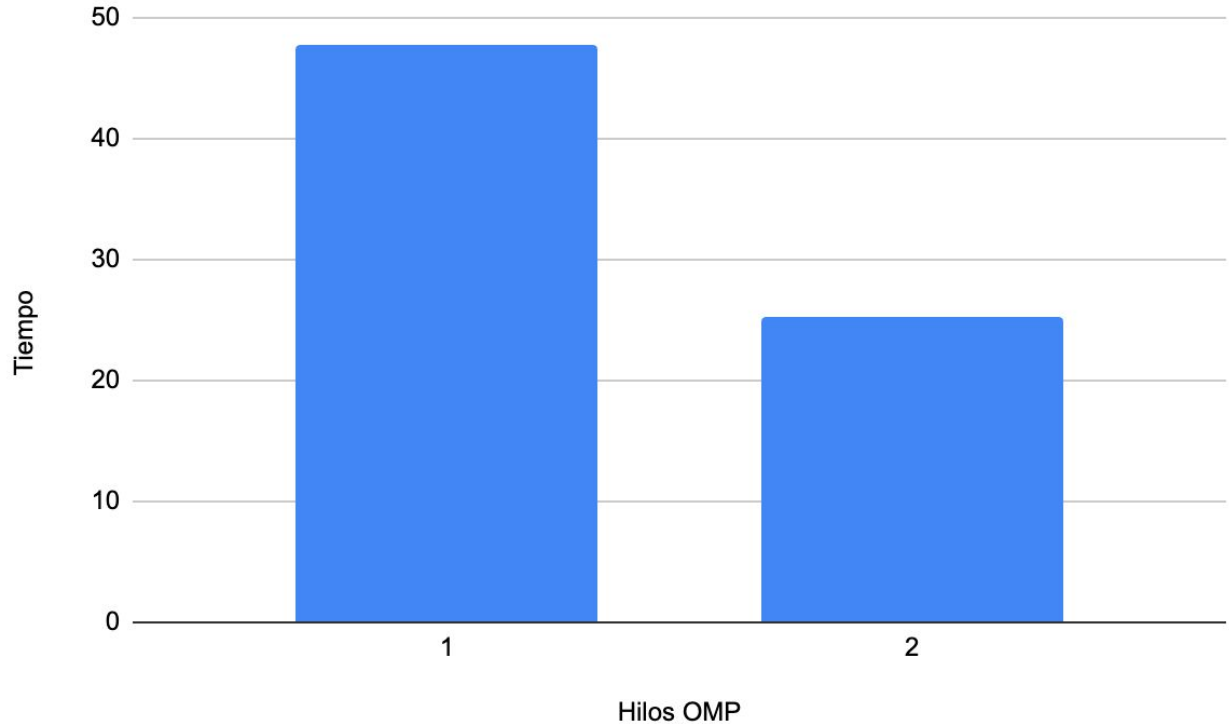
```
#pragma omp parallel for ←  
for (long long i = 0; i < total_combinaciones; i++) {  
    char cadena[LONGITUD_CONTRASENA + 1];  
    indice_a_cadena(i, LONGITUD_CONTRASENA, cadena);  
    if (strcmp(cadena, CONTRASENA) == 0) {  
        #pragma omp critical ←  
        {if (encontrado == 0) {  
            encontrado = 1;  
            strcpy(resultado, cadena);  
        }  
    }
```

**Cambios y agregaciones
en el código**

**Un solo hilo ejecutando
evita sobrescribir
“encontrado”.**

OpenMP (Tiempos)

Utilizando el doble de hilos el tiempo se ve reducido casi a la mitad del tiempo



OpenMP (Tiempos)

Optimización	OpenMP
O0	51,6069382
O1	27,8930292
O2	25,4547516
O3	25,2086643

Cálculo de parámetros (OpenMP)

OpenMP + O3 vs. Secuencial + O0

Aceleración obtenida	Eficiencia
$\frac{\textit{Tiempo secuencial}}{\textit{Tiempo Optimizado}}$	$\frac{\textit{Tiempo secuencial}}{\textit{cores}}$
104,5159355 / 25,2086643 = 4,146032239	4,146032239 / 2 = 2,073016119 (o 207,301611%)

MPI (Código)

```
int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);
    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if (rank == 0) {
        printf("PROCESOS MPI: %d\n", size);
        printf("BUSCANDO CONTRASENA: '%s' CON LONGITUD: %d, CARACTERES POSIBLES:
%d)\n",
            CONTRASENA, LONGITUD_CONTRASENA, NUM_CAR);
        printf("MEDICION CON MPI_Wtime (segundos)\n");
        long long total_combinaciones = potencia_entera(NUM_CAR, LONGITUD_CONTRASENA);
    }
```

**Ejecutado por
un solo
proceso
if (rank == 0)**

```
    if (rank == 0) {
        printf("POSIBLES COMBINACIONES: %lld\n\n", total_combinaciones);
    }

    int encontrado = 0;
    char resultado[LONGITUD_CONTRASENA + 1];

    int encontrado_global = 0;
    char resultado_global[LONGITUD_CONTRASENA + 1];
    resultado_global[0] = '\0';
```

MPI (Código)

```
long long inicio = (total_combinaciones / size) * rank;
long long final;
if (rank == size - 1) {
    final = total_combinaciones;
}
else {
    final = inicio + (total_combinaciones / size);
}
encontrado = 0;
```

**Reparte ejecución del
bucle entre procesos**

```
MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

for (long long i = inicio; i < final; i++) {
    char cadena[LONGITUD_CONTRASENA + 1];
    indice_a_cadena(i, LONGITUD_CONTRASENA, cadena);
    if (strcmp(cadena, CONTRASENA) == 0) {
        if (encontrado == 0) {
            encontrado = 1;
            strcpy(resultado, cadena);
        }
    }
}
```

MPI (Código)

```
double end = MPI_Wtime();  
double tiempo_local = end - start;
```

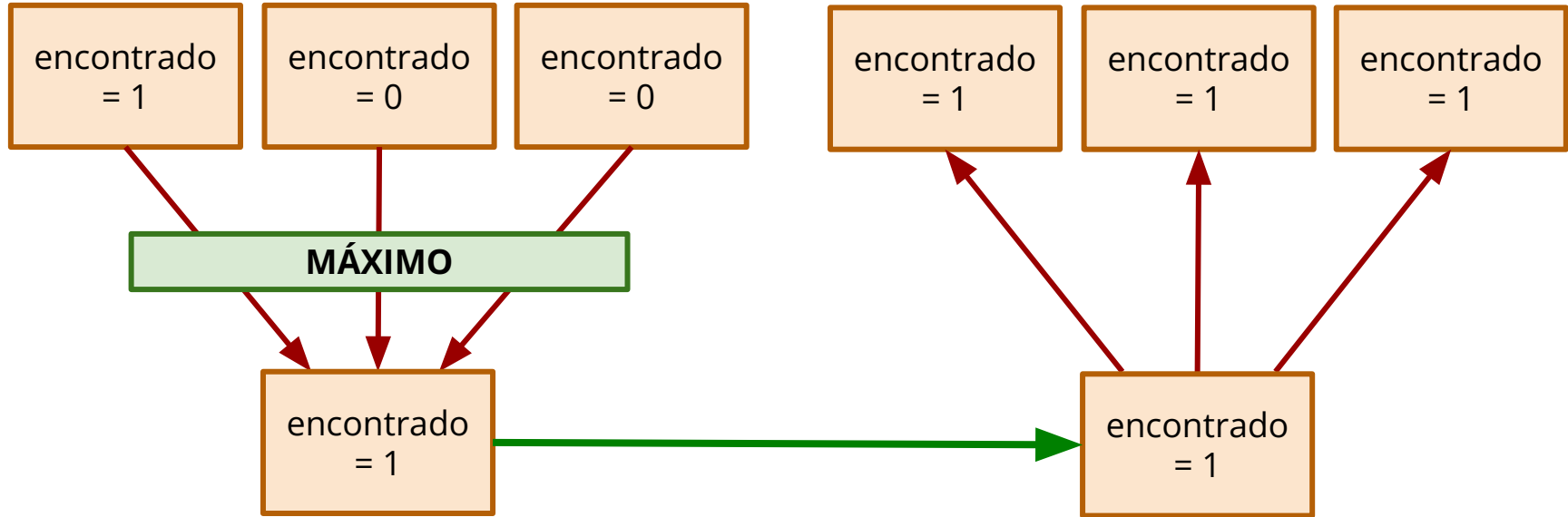
```
double tiempo_global = 0.0;  
MPI_Reduce(&tiempo_local, &tiempo_global, 1, MPI_DOUBLE, MPI_MAX, 0,  
MPI_COMM_WORLD);
```

```
//si alguno encuentra la contraseña, encontrado=1, debe avisar a los demas  
MPI_Allreduce(&encontrado, &encontrado_global, 1, MPI_INT, MPI_MAX,  
MPI_COMM_WORLD);
```

```
//si algun proceso la encontro (y se lo ha comunicado a los demas procesos)  
if (encontrado_global) {  
    // el proceso que lo encontro (que no sea el 0) se lo envia al 0  
    if (encontrado && rank != 0) {  
        MPI_Send(resultado, LONGITUD_CONTRASENA + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);  
    }  
}
```

MPI (Código)

ALL REDUCE (entre 3 procesos)



MPI (Código)

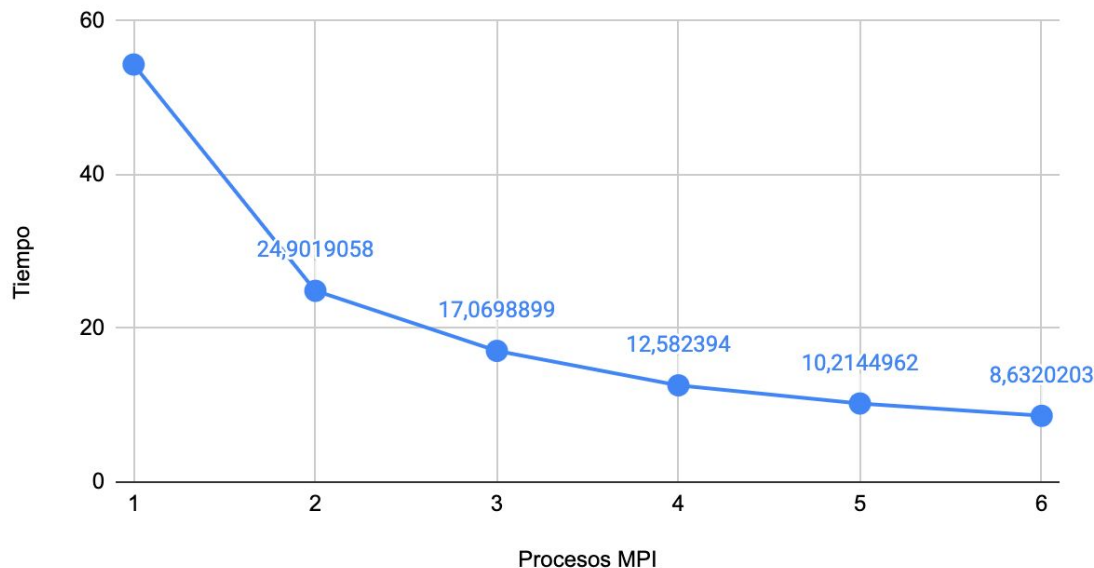
```
// El proceso 0 recibe la contraseña encontrada
if (rank == 0) {
    if (encontrado) {
        strcpy(resultado_global, resultado); // caso en el que el proceso 0 es el que la encuentra
    }
    else {
        MPI_Status status;
        MPI_Recv(resultado_global, LONGITUD_CONTRASENA + 1, MPI_CHAR,
            MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
    }
    printf("CONTRASENA ENCONTRADA: '%s'\n", resultado_global);
    printf("TIEMPO MPI (s, max entre procesos): %.6f\n", tiempo_global);
}
}
else if (rank == 0) {
    printf("NO SE ENCONTRO\n");
    printf("TIEMPO MPI (s, max entre procesos): %.6f\n", tiempo_global);
}

MPI_Finalize();
```

MPI (Tiempos)

El tiempo es aproximadamente proporcional al número de procesos MPI que se usen

Tiempo en base a procesos



MPI (Tiempos)

Optimización	MPI
O0	18,0479056
O1	9,5458501
O2	8,9613405
O3	8,6320203

Cálculo de parámetros (MPI)

MPI + O3 vs. Secuencial + O0

Aceleración obtenida

$$\frac{\textit{Tiempo secuencial}}{\textit{Tiempo Optimizado}}$$

$$104,5159355 / 8,6320203 = \mathbf{12,107934412}$$

Eficiencia

$$\frac{\textit{Tiempo secuencial}}{\textit{cores}}$$

$$12,107934412 / 6 = \mathbf{2,017989068} \text{ (o } 201,798906\%)$$

OpenMP + MPI (código)

```
int main(int argc, char* argv[]) {
    MPI_Init(&argc, &argv);

    int size, rank;
    MPI_Comm_size(MPI_COMM_WORLD, &size);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);

    int hilos_por_proceso = omp_get_max_threads();

    omp_set_num_threads(hilos_por_proceso);

    if (rank == 0) {
        printf("PROCESOS MPI: %d\n", size);
        printf("HILOS OpenMP POR PROCESO: %d\n", hilos_por_proceso);
        printf("TOTAL HILOS LANZADOS (ranks x hilos): %d\n", size * hilos_por_proceso);
        printf("BUSCANDO CONTRASENA: '%s' CON LONGITUD: %d, CARACTERES POSIBLES: %d\n",
               CONTRASENA, LONGITUD_CONTRASENA, NUM_CAR);
        printf("MEDICION CON MPI_Wtime (segundos)\n");
    }

    long long total_combinaciones = potencia_entera(NUM_CAR, LONGITUD_CONTRASENA);
```

OpenMP + MPI (código)

```
if (rank == 0) {  
    printf("POSIBLES COMBINACIONES: %lld\n\n", total_combinaciones);  
}  
int encontrado = 0;  
char resultado[LONGITUD_CONTRASENA + 1];  
int encontrado_global = 0;  
char resultado_global[LONGITUD_CONTRASENA + 1];  
resultado_global[0] = '\0';  
  
//dividir que proceso hace cada parte del bucle  
long long inicio = (total_combinaciones / size) * rank;  
  
long long final;  
if (rank == size - 1) {  
    final = total_combinaciones;  
}  
else {  
    final = inicio + (total_combinaciones / size);  
}  
  
encontrado = 0;  
resultado[0] = '\0';
```

OpenMP + MPI (código)

```
MPI_Barrier(MPI_COMM_WORLD);
double start = MPI_Wtime();

#pragma omp parallel for
for (long long i = inicio; i < final; i++) {
    char cadena[LONGITUD_CONTRASENA + 1];
    indice_a_cadena(i, LONGITUD_CONTRASENA, cadena);
    if (strcmp(cadena, CONTRASENA) == 0) {
        #pragma omp critical
        {
            if (encontrado == 0) {
                encontrado = 1;
                strcpy(resultado, cadena);
            }
        }
    }
}

double end = MPI_Wtime();
double tiempo_local = end - start;
```

OpenMP + MPI (código)

```
double tiempo_global = 0.0;
MPI_Reduce(&tiempo_local, &tiempo_global, 1, MPI_DOUBLE, MPI_MAX, 0, MPI_COMM_WORLD);

//si alguno encuentra la contraseña, encontrado=1, debe avisar a los demas
MPI_Allreduce(&encontrado, &encontrado_global, 1, MPI_INT, MPI_MAX, MPI_COMM_WORLD);

//si algun proceso la encontro (y se lo ha comunicado a los demas procesos)
if (encontrado_global) {
    // el proceso que lo encontro (que no sea el 0) se lo envia al 0
    if (encontrado && rank != 0) {
        MPI_Send(resultado, LONGITUD_CONTRASENA + 1, MPI_CHAR, 0, 0, MPI_COMM_WORLD);
    }
}
```

OpenMP + MPI (código)

```
// El proceso 0 recibe la contraseña encontrada
if (rank == 0) {
    if (encontrado) {
        strcpy(resultado_global, resultado); // caso en el que el proceso 0 es el que la encuentra
    }
    else {
        MPI_Status status;
        MPI_Recv(resultado_global, LONGITUD_CONTRASENA + 1, MPI_CHAR,
                 MPI_ANY_SOURCE, 0, MPI_COMM_WORLD, &status);
    }
    printf("CONTRASENA ENCONTRADA: '%s'\n", resultado_global);
    printf("TIEMPO MPI+OpenMP (s, max entre procesos): %.6f\n", tiempo_global);
}
}

else if (rank == 0) {
    printf("NO SE ENCONTRO\n");
    printf("TIEMPO MPI+OpenMP (s, max entre procesos): %.6f\n", tiempo_global);
}

MPI_Finalize();
return 0;
```

OpenMP + MPI

Optimización	OpenMP + MPI
O0	18,6610845
O1	9,3895018
O2	8,9035469
O3	8,517027

Cálculo de parámetros (OpenMP + MPI)

OpenMP + MPI + O3 vs. Secuencial + O0

Aceleración obtenida

$$\frac{\textit{Tiempo secuencial}}{\textit{Tiempo Optimizado}}$$

$$104,5159355 / 8,517027 = \mathbf{12,271410658}$$

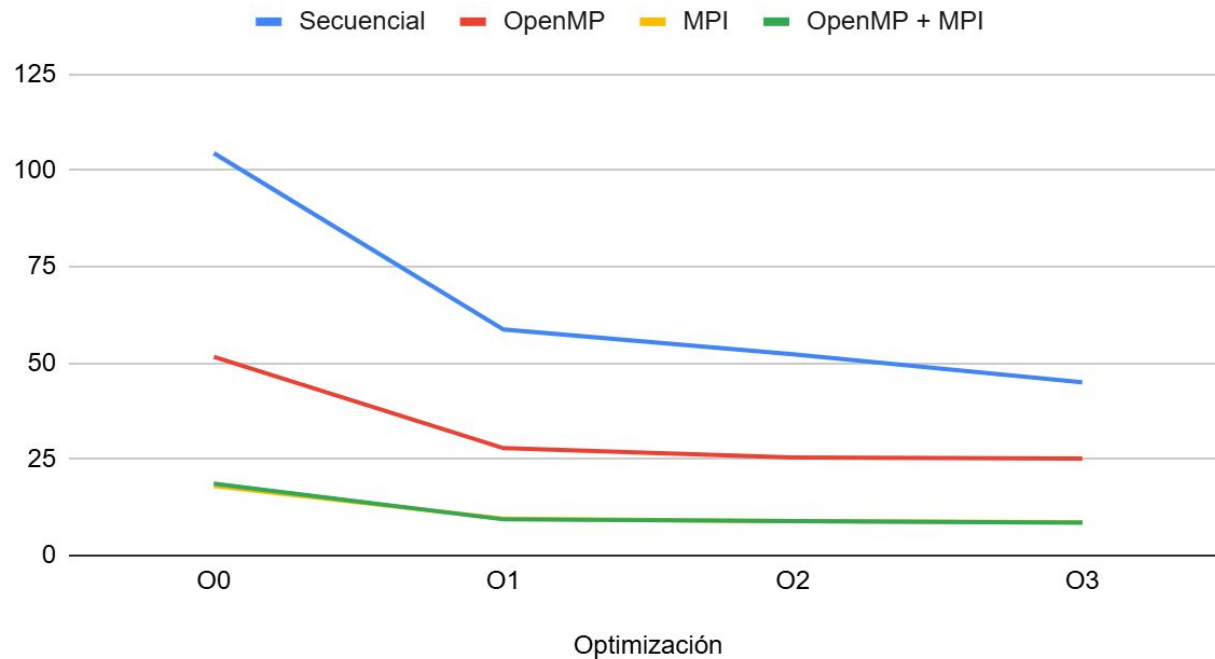
Eficiencia

$$\frac{\textit{Tiempo secuencial}}{\textit{cores}}$$

$$12,271410658 / 12 = \mathbf{1,022617554} \text{ (o } 102,261755\%)$$

Resultados

Secuencial, OpenMP, MPI y OpenMP + MPI



Conclusiones

- Mejor optimización O3: eliminación de overhead + reducción de accesos a memoria (cuello de botella).
- MPI más eficiente que openMP
- OpenMP + MPI no mejora mucho (por memory-bound)
- Impacto del overhead
- invertir en más cores es inútil sin antes reducir los accesos a memoria o aumentar la intensidad aritmética.